[OTA] Technical specs for artifact packaging, OTA Agent & Bootloader

Introduction

As described in the scope document, this node operates in coordination with the Master node to receive different types of artifacts and manage their lifecycle. Its responsibilities include validating artifact integrity and compatibility, handling artifact distribution, installation, and activation, and performing other related coordination tasks. This document further elaborates on the internal design and functional responsibilities through which these tasks are executed.

In the ideal deployment model, a single instance of the daemon runs on each computing device. This instance acts as a subordinate component in the OTA process, ensuring that updates are installed and activated correctly. In addition, it is responsible for local rollback handling, system health monitoring, partition management (active/inactive), and reporting status and progress updates back to the Master node.

Caveat:

High-Performance Computing units (HPCs) on which the Master OTA service is running are not exempt from update operations. These nodes will also host an OTA Agent instance to perform software and firmware updates on the same device. In such scenarios, the agent must support self-update capabilities while maintaining system stability and availability.

For firmware updates on the HPC system, temporary delegation of certain orchestration responsibilities will be granted to the secondary master service on other HPC for more details check the Master node design.

Artifact types:

Before detailing the implementation, this section first defines the supported artifact types and their distribution structures. Installation, activation, and rollback mechanisms vary depending on the artifact type; therefore, each category and its corresponding flow is described separately.

1. Software artifacts

Software artifacts do not require a full reset of the ECU or HPC for activation. The system supports installation, activation, and rollback of these artifacts with minimal downtime. Any impact is limited to the specific service or feature associated with the artifact, while the rest of the system remains operational.

2. Firmware artifacts

Firmware artifacts require a reset of the entire ECU or HPC to complete activation. Installation is performed in an inactive or staging area, followed by activation during a controlled reset cycle. Rollback mechanisms are supported to ensure recovery in case of failure, with downtime limited to the features or services hosted on the affected device.

Packaging and distribution:

The packaging structure (archive) will mostly the same for both artifact types, there will some differences between the packaging which will be highlighted below,

Manifest additions:

- `package_identifier` : this will define the unique ID of the application.
- version: version of the application (we can consider it as `const char *` and let the developer decides its versioning conventions)

- `node_identifier` : For which type of node, this package is for, there will be a master level validation to check if the node support this type of package
- `minimum_os_version` : defines what will be the minimum OS version, this application is for. This will help to prevent compatibility issues with the OS
- `maximum_os_version` : defines what will be the maximum OS version, this application is for. This will help to prevent compatibility issues with the OS.
- `minimum_disruptsdk_version` : defines what will be the minimum DisruptSDK version, this application is for. This will help to prevent compatibility issues with the ecosystem
- `minimum_disruptsdk_version` : defines what will be the maximum DisruptSDK version, this application is for. This will help to prevent compatibility issues with the ecosystem.
- `targetted_region` : this will define whether the application is supported in the region (a rule that can be handled by the OEM)
- `custom_oem_rules` : to define the functionality by the OEM for different variants of the same car.

Addition of `/libs` directory:

For software application artifacts, a dedicated and fixed directory named `/libs` shall be included within the package. This directory contains all custom and third-party shared libraries required by the application to operate correctly. Again, as I mentioned only the custom and third-party shared libraries will be included, there will be a common libraries set which will serve from the root directory's lib only.

The introduction of an application-scoped `/libs` directory enables isolation of shared libraries on a per-application basis, thereby preventing version discrepancies and dependency conflicts between applications deployed on the same ECU or HPC.

Benefits

- Library Version Isolation: Each application carries its own library dependencies, eliminating conflicts arising from incompatible shared library versions across different applications.
- Predictable Runtime Behavior: By explicitly defining the libraries used by an application, runtime behavior remains consistent across installations, updates, and rollbacks.
- Simplified Rollback Mechanism: Rolling back an application automatically restores the exact library set associated with that version, without impacting other applications or system-wide libraries.
- Reduced System-Wide Coupling: Applications are decoupled from global library updates, minimizing the risk of unintended regressions caused by changes to shared system libraries.
- Safer Incremental Updates: Updates to one application do not require validation of unrelated applications, as library changes remain scoped to the updated package.
- Improved Security and Auditability: Bundling dependencies allows precise tracking of library versions used by each application, facilitating vulnerability analysis, compliance audits, and targeted patching.
- Easier Cross-Platform and Cross-ECU Portability: Applications become more portable across different ECUs or HPC configurations, as required dependencies are self-contained within the package.

Addition of `/res` directory:

For software application artifacts, an optional directory named `/res` may be included within the package. This directory is intended to contain non-executable application resources such as configuration files, templates, static assets, localization files, and other runtime data required by the application.

The `/res` directory enables clear separation between executable logic, shared libraries, and application-specific resources. This separation improves maintainability, simplifies updates and rollbacks, and allows resource-level

changes without impacting application binaries or library dependencies.

Fixing the executable name `/exe`:

For software and firmware artifacts, a fixed and well-defined executable named `/exe` shall be provided at the root of the package. This standardization ensures a consistent and predictable entry point for application startup, activation, and supervision by the OTA Agent and runtime management components.

> There will one more fixed file called rdinit but this will only applicable in case of firmware artifacts

How the installation will work by the agent passively?

Software Artifacts

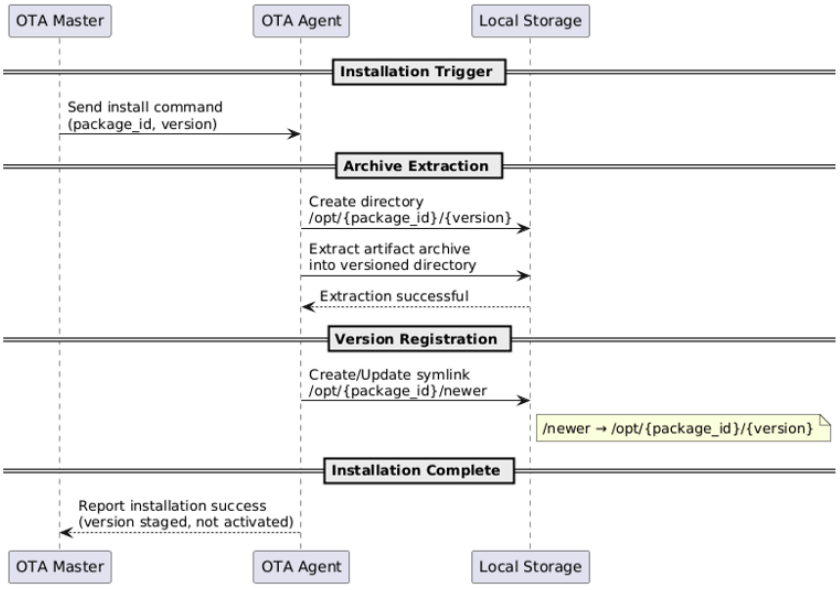
Upon receiving an installation directive from the OTA Master, the OTA Agent initiates the installation process for the specified software artifact. The agent extracts the artifact archive into a versioned directory at the following location:

```
/opt/{package_id}/{version}
```

After successful extraction and validation, the agent creates or updates a symbolic link named `newer` under the package root:

```
/opt/{package_id}/newer → /opt/{package_id}/{version}
```

This symbolic link represents the latest installed version pending activation. By using an indirection layer through the `newer` symlink, the system enables controlled activation, simplified rollback, and coexistence of multiple versions without directly impacting the currently active application instance.



Firmware Artifacts

The firmware installation process is highly dependent on the underlying bootloader implementation and the target hardware platform. This dependency arises because the layout, management, and semantics of inactive (A/B or equivalent) slots vary across bootloaders and are often manufacturer-specific.

As a result, the OTA Agent must rely on bootloader-provided mechanisms and platform-specific update procedures to correctly stage, activate, and roll back firmware artifacts. The agent does not assume a uniform slot layout. Instead, it integrates with the capabilities exposed by the bootloader on the target system.

Below are two representative examples illustrating how firmware installation differs across platforms:

NVIDIA Orin Platform:

On NVIDIA Orin–based platforms, firmware updates are integrated with the existing boot chain and `extlinux (or similar)` configuration. During installation, the OTA Agent copies the kernel image and the corresponding initramfs (`rdinit`) into the `extlinux` directory.

A preconfigured boot entry, referred to as the `newer` slot, is used as the default inactive slot for staging updates. This slot is defined in advance within the bootloader configuration and includes built-in safety mechanisms such as boot retry counters, automatic fallback to the previous stable slot, and sanity checks to detect boot failures.

U-Boot–Based Platform:

On U-Boot–based platforms, firmware updates are performed using predefined storage partitions and U-Boot environment variables. The OTA Agent writes the firmware components (such as kernel image, device tree blob, and optional initramfs) to the designated inactive partition as defined by the platform layout.

During the installation phase, the OTA Agent may read (but not modify) standard U-Boot environment variables in order to determine the inactive slot and its associated storage location. Commonly used variables include:

- `bootcmd` – Defines the boot command sequence (read-only during installation)
- `bootargs` – Kernel command-line parameters (read-only during installation)
- `bootcount` – Current boot attempt counter (read-only during installation)
- `bootlimit` – Maximum allowed boot attempts before fallback (read-only during installation)
- `mmcdev` , `mmcpart` – Active MMC device and partition identifiers
- `mmcdev_alt` , `mmcpart_alt` *(if defined)* – Alternate device or partition identifiers

The OTA Agent uses these variables solely to identify the correct inactive partition for staging firmware. No modification is performed on variables that affect boot flow, slot selection, retry logic, or activation behavior.

MCUBoot-Based Platform:

On MCUboot-based platforms, firmware updates are performed using a slot-based flash layout defined by the bootloader. The firmware is delivered as a single monolithic binary containing the application, RTOS kernel, and supporting components.

The OTA Agent executes as part of the running firmware and stages updates by writing the new image to the inactive MCUboot slot as defined in the platform flash map.

During the installation phase, the OTA Agent may read (but not modify) MCUboot-defined metadata to identify the inactive slot and validate image state. Commonly used MCUboot structures and fields include:

- **Primary slot (** `slot0` **)** – Currently executing firmware image
- **Secondary slot (** `slot1` **)** – Inactive slot used for staging updates
- **Image header (** `struct image_header` **)** – Contains image size, version, and flags
- **Image trailer (** `struct image_trailer` **)** – Contains bootloader-managed state fields
- `image_ok` **flag** – Indicates a confirmed image
- `swap_type` / `swap_status` – Indicates pending, permanent, or revert swap state

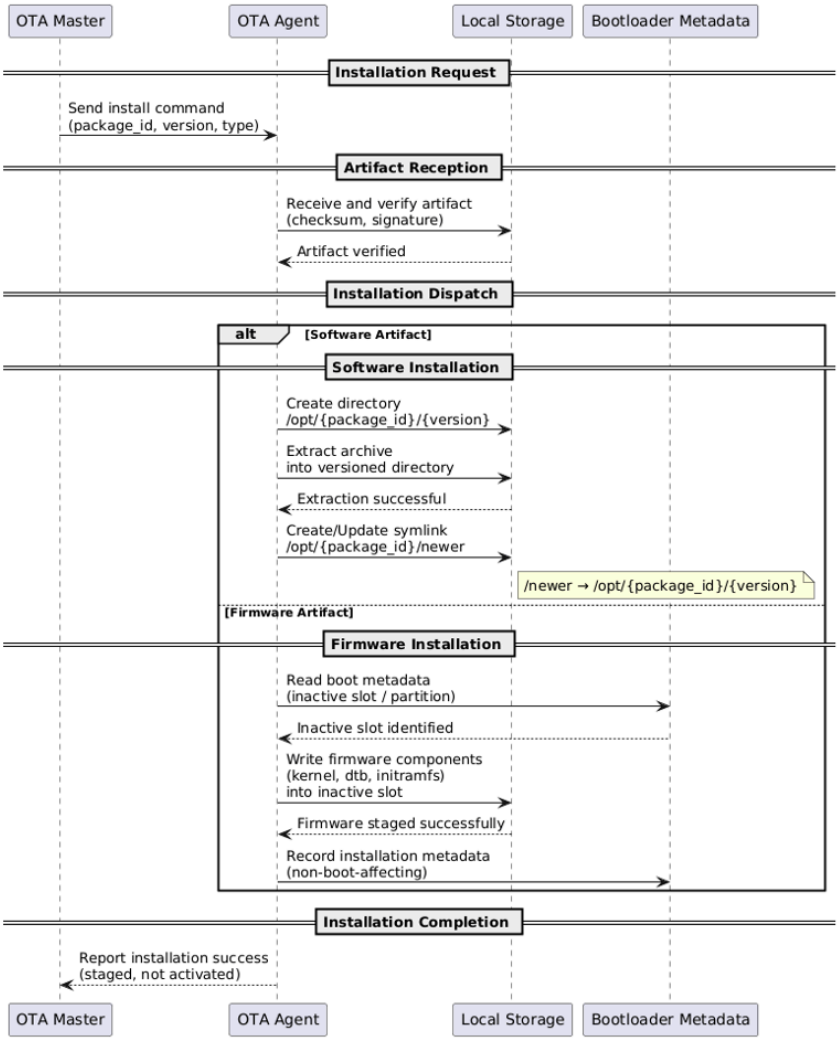
The OTA Agent uses this information solely to ensure correct placement of the firmware image and to request activation by marking the staged image as pending using MCUboot-defined mechanisms. The agent does not alter MCUboot swap logic, flash layout, or rollback policy.

Final image selection, swap execution, retry handling, and rollback decisions are exclusively enforced by MCUboot during the subsequent boot cycle.

Single application based operating systems:

In case of single binary operating systems, firmware artifacts are delivered and managed as monolithic images rather than file-based software components and the OTA Agent is implemented as an integral module within the firmware image itself and will be receiving and writing the updated image in inactive slot.

Upon activation signal, the same agent will also update the bootloader arguments to start with boot slot B with max number of retries, etc.



How the activation will work by the agent?

Activation is a controlled and explicitly authorized phase of the OTA lifecycle and is always initiated by the OTA Master. The OTA Agent does not autonomously activate any artifact; it acts only upon receiving a validated activation command from the Master and after all prerequisite safety conditions are satisfied.

Software Artifact

For software artifacts, activation is performed by atomically updating version reference links within the application installation directory. The OTA Agent first updates the existing symbolic link:

```
/opt/{package_id}/current
```

to point to the previously active version by replacing or updating:

```
/opt/{package_id}/prev
```

Next, the OTA Agent updates the `current` symbolic link to point to the version staged under:

```
/opt/{package_id}/newer
```

As a result, the active application version becomes:

```
/opt/{package_id}/current → /opt/{package_id}/{version}
```

while the previously active version is preserved as:

```
/opt/{package_id}/prev → /opt/{package_id}/{previous_version}
```

After the symbolic links are updated, the OTA Agent restarts the corresponding service so that execution begins using the newly activated version.

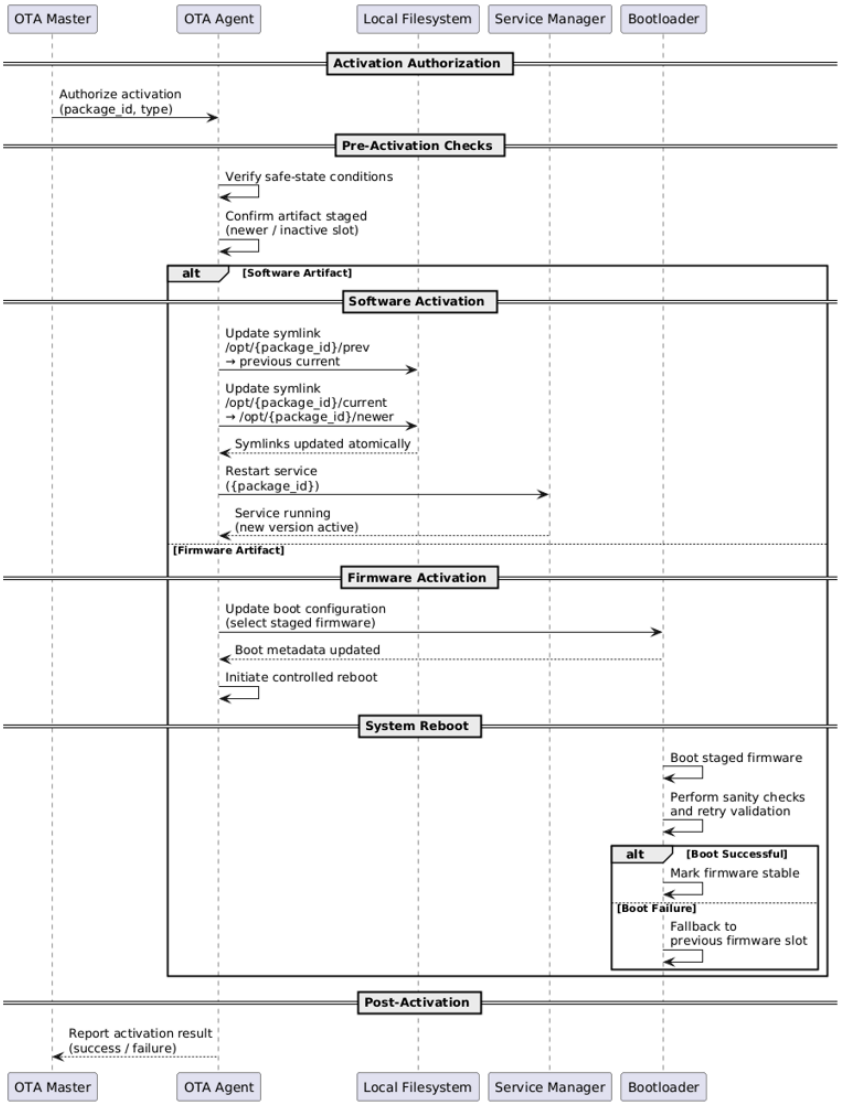
This activation mechanism enables atomic version switching, minimizes service downtime, and ensures that a known-good version is always retained for immediate rollback.

Firmware Artifact

For firmware artifacts, activation is performed through a controlled reboot sequence and is tightly coupled with the bootloader and platform-specific boot chain. Unlike software artifacts, firmware activation cannot occur at runtime and requires a system reset to transfer execution to the newly staged firmware.

Once the activation request is authorized by the OTA Master and all safety conditions are met, the OTA Agent initiates a controlled reboot. During the subsequent boot, the bootloader transfers control to the staged firmware image and applies built-in validation mechanisms such as boot attempt counters, sanity checks, and watchdog supervision.

If the firmware boots successfully and reaches a defined post-boot confirmation point, the bootloader promotes the new firmware to a stable state. In case of repeated boot failures or unmet sanity conditions, the bootloader automatically falls back to the previously active firmware slot without requiring OTA Agent intervention during early boot.



How the activation failures will be handled?

Activation failures are handled differently for software and firmware artifacts due to their distinct execution and recovery characteristics. In all cases, failure handling is deterministic, bounded, and coordinated with the OTA Master.

Software Artifact

For software artifacts, activation failures may occur during service restart, initialization, or early runtime validation. The OTA Agent detects such failures through service manager feedback, health checks, or watchdog timeouts.

Upon detecting a failure:

1. The OTA Agent immediately reverts the symbolic links to restore the previously active version:

```
/opt/{package_id}/current → /opt/{package_id}/prev
```

2. The affected service is restarted using the restored version.
3. The failed version remains staged but inactive for further inspection or cleanup.
4. The OTA Agent reports the failure, rollback action, and diagnostic information to the OTA Master.

This approach ensures rapid recovery with minimal service downtime and no impact on unrelated applications.

Firmware Artifact

For firmware artifacts, activation failures are handled primarily by the bootloader during early boot, as the OTA Agent is not active at this stage.

Failure conditions may include:

- Failure to boot the staged firmware
- Repeated crashes or watchdog resets
- Failed sanity or health checks defined by the platform

In such cases:

1. The bootloader automatically detects the failure using retry counters, watchdogs, or validation hooks.
2. The system falls back to the previously active firmware slot without OTA Agent intervention.
3. Once the system boots successfully into the fallback firmware, the OTA Agent resumes operation.
4. The OTA Agent reports the activation failure and fallback outcome to the OTA Master.

No manual intervention is required to restore system operability.

Reporting and Recovery Coordination

In all failure scenarios, the OTA Agent:

- Reports activation status, rollback actions, and failure diagnostics to the OTA Master
- Marks the failed artifact as invalid or blocked for reactivation
- Awaits further instructions from the Master for retry, cleanup, or blacklisting

This centralized coordination ensures consistent policy enforcement, traceability, and safe system evolution across the fleet.

Technical Interaction and Integration Between OTA Agent and Bootloader

The OTA Agent exposes a generic, bootloader-agnostic interface that abstracts all bootloader-specific operations. This interface defines a common set of functions required for firmware installation, activation preparation, and status querying, without embedding any platform-specific logic within the core OTA Agent.

Bootloader-specific behavior is implemented through dedicated plugins that conform to this interface. Each plugin encapsulates the details required to interact with a particular bootloader and hardware platform, such as how inactive slots are populated, how boot metadata is updated, and how platform-specific constraints are handled (for example, slot management on NVIDIA Orin platforms or partition and environment handling on NXP U-Boot–based systems).

This plugin-based approach ensures that all bootloader-specific changes remain isolated within their respective plugins, allowing the OTA Agent core to remain stable, maintainable, and portable across platforms. New bootloader or hardware support can be added by introducing a new plugin implementation without requiring changes to the OTA Agent's core logic.